rittman analytics

INTERNAL ENABLEMENT · MAY 2026

wire Framework Developer Onboarding.

The delivery framework that makes AI-assisted consulting consistent, contextual, and auditable across every engagement.

Rittman Analytics | Engineering Enablement

V3.4.20

T: +44 (0)208 058 5951

W: rittmananalytics.com

E: info@rittmananalytics.com

rittman analytics

AGENDA

What we will cover today.

01	What Wire is and why it exists	10 MIN
02	Installation and your first engagement	15 MIN
03	The artifact lifecycle — generate, validate, review	25 MIN
04	Release types — choosing the right one	20 MIN
05	Automatic context and state management	15 MIN
06	Skills — development outside Wire commands	15 MIN
07	Autopilot and advanced features	10 MIN
08	Q&A and next steps	10 MIN

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

PART ONE

What Wire is, and why it exists.

Two problems with AI-assisted delivery — and one framework that fixes both.

01

T: +44 (0)208 058 5951

W: rittmananalytics.com

E: info@rittmananalytics.com

rittman analytics

THE PROBLEM

Two problems with AI-assisted delivery.

Drop a senior engineer into a generative tool with no scaffolding and you get a different shaped artifact every time — and a fresh amnesia at the start of every conversation.

Inconsistency.

> *"Write me a dbt model for this table."*

Gets you something that might not follow naming conventions, might not have the right tests, might not fit your three-layer architecture.

Context loss.

Every new conversation starts from scratch. The AI doesn't know what was decided last week, what the client's constraints are, or where you are in the project.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

THE SOLUTION

Wire solves both problems.

Specs make the AI follow our conventions. State files make every conversation pick up where the last one left off.

Consistency.

Every command reads a spec before generating anything. The spec tells the AI exactly which conventions to follow, which upstream inputs to read, and how to validate output.

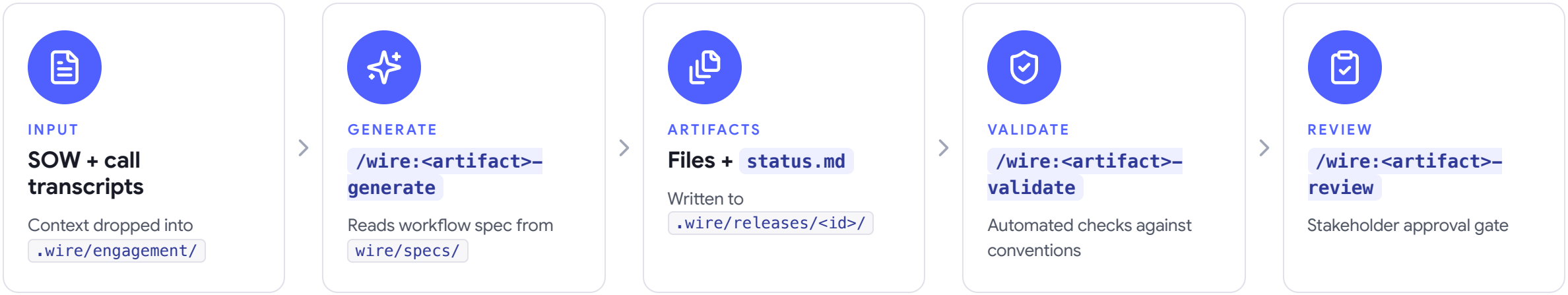
Context.

State is tracked in `status.md` and `execution_log.md`. The `engagement-context` skill reloads this state automatically at the start of every conversation.

rittman analytics

ARCHITECTURE

The whole framework, in one diagram.



T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

BY THE NUMBERS

What you get out of the box.

67

COMMANDS

Covering six delivery phases.

7

RELEASE TYPES

From discovery sprint to agentic commerce.

16

SKILLS

Auto-activate during day-to-day development.

0

SETUP STEPS

Install plugin · `/wire:new` · start generating.

T: +44 (0)208 058 5951

W: rittmananalytics.com

E: info@rittmananalytics.com

rittman analytics

PART TWO

Installation and first engagement.

Two commands to install, one command to spin up your first release.

T: +44 (0)208 058 5951

W: rittmananalytics.com

E: info@rittmananalytics.com

02

rittman analytics

INSTALLATION

Four commands. Then you're done.

1 · Add the marketplace

```
/plugin marketplace add rittmananalytics/wire-plugin
```

2 · Install the plugin

```
/plugin install wire@rittman-analytics
```

3 · Reload plugins

```
/reload-plugins
```

4 · Verify

```
/wire:help
```

What "done" looks like

You should see the full Wire command tree — six delivery phases, generate / validate / review triples under each.

```
$ /wire:help

Wire Framework v3.4.20
──────────────────────────────────

  discovery     6 commands
  requirements  3 commands
  pipeline      9 commands
  dbt           12 commands
  semantic      6 commands
  dashboards    8 commands
  release       6 commands
  meta          17 commands

Run /wire:new to create an
engagement.
```


T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

BOOTSTRAP

Creating an engagement.

Run one command. Wire prompts you for everything it needs to spin up a release.

```
/wire:new
```

It is interactive — answer the prompts in chat, no flags to memorise.

What it asks you

- **Client and engagement name**
- **Repo mode** — combined, or dedicated delivery repo
- **Release type** — pick from the seven
- **Release ID** — e.g. `01-discovery` or `01-client-platform`
- **SOW path** — optional, copied to `engagement/sow.md`
- **Issue tracker** — Jira, Linear, or none

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

SCAFFOLDING

What `/wire:new` writes to disk.

```
.wire/
  engagement/
    context.md      ← fill with client details
    sow.md          ← copy of the SOW
  releases/
    01-your-release/
      status.md          ← process definition
      execution_log.md   ← empty; fills as you work
```

The shape of `status.md`

Frontmatter lists every in-scope artifact, each starting at `not_started`.

This is the process definition for the entire release — it's the source of truth for what's queued, what's mid-flight, and what's blocked.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

PART THREE

The artifact lifecycle.

Three gates, every artifact: generate → validate → review.

T: +44 (0)208 058 5951

W: rittmananalytics.com

E: info@rittmananalytics.com

03

rittman analytics

LIFECYCLE

Three gates, every artifact.

GATE 1

Generate

AI reads spec + upstream inputs, produces the artifact. Files land in the release folder.

GATE 2

Validate

Automated checks against conventions. **PASS** or **FAIL** with specific failures listed.

GATE 3

Review

You or the stakeholder approves. `approved` or `changes_requested`.

Each gate updates `status.md` and appends to `execution_log.md`. Nothing happens off the books.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

VALIDATE · SAMPLE OUTPUT

What a validation report tells you.

```
## Validation Report — requirements

✅ 13/13 checks passed

Checks run:
✅ All 13 sections present
✅ Each deliverable has acceptance criteria
✅ Non-functional requirements
     (performance, security, freshness)
✅ D1–D5 mapped to framework artifacts
✅ Timeline feasibility check
```

WHY THIS MATTERS

Specific failures, every time.

If a check fails, the output names the specific failure — the section that's missing, the deliverable without acceptance criteria, the timeline that doesn't add up.

You know exactly what to fix before you re-run.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

HANDS-ON · EXERCISE 3

Full requirements lifecycle, end-to-end.

Add context to `.wire/engagement/context.md`, then run the full requirements lifecycle on the training release.

```
/wire:requirements-generate 01-training-demo
/wire:requirements-validate 01-training-demo
/wire:requirements-review   01-training-demo
```

Read `execution_log.md` after each command. Notice how the row appended differs by gate.

What you should see

- A requirements document with 13 sections
- `status.md` moves `requirements` from `not_started` → `approved`
- Three new rows in `execution_log.md`
- The next artifact unlocked in the queue

25 min · work solo, ask for help in `#wire-framework`

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

PART FOUR

Release types.

Seven shapes of engagement, each with its own artifact queue.

T: +44 (0)208 058 5951

W: rittmananalytics.com

E: info@rittmananalytics.com

rittman analytics

RELEASE TYPES

Pick the shape that matches the engagement.

TYPE	WHEN TO USE
`discovery`	Uncertain scope — scoping sprint before delivery.
`full_platform`	End-to-end: SOW → production dashboards.
`pipeline_only`	New pipeline + dbt, no BI layer.
`dbt_development`	dbt only, data already in the warehouse.
`dashboard_extension`	New dashboards on an existing semantic layer.
`dashboard_first`	Early prototyping with seed data, before client data arrives.
`enablement`	Training and docs for an existing platform.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

DISCOVERY RELEASE

The Shape Up flow, mapped onto Wire.

Run a discovery release when the client's scope is unclear, or when you need formal sign-off on *what* to build before building it.

Then run `/wire:release:spawn` to create downstream delivery releases.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

DASHBOARD-FIRST RELEASE

Prototype with seed data, swap to real sources later.

Useful when client data access is delayed but stakeholders want to see something working.

```
requirements → mockups → viz_catalog → data_model
                                            ↓
                                       seed_data  ← AI generates realistic CSV fixtures
                                            ↓
                                  dbt (uses ref('seed')) → semantic_layer → dashboards
                                            ↓ (when client data arrives)
                                       data_refactor  ← transitions to real sources
```


T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

HANDS-ON · EXERCISE 4

Release type selection.

Three SOW descriptions are pinned in `#wire-framework`. Read each and decide:

1. The correct release type
2. Two in-scope artifacts
3. One artifact that would be `not_applicable`

Format

Work in pairs. One of you names the type and defends it; the other plays devil's advocate on which artifacts should drop out.

Bring your three answers back to the room — we'll compare against the answer key.

10 min · work in pairs

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

PART FIVE

Automatic context and state management.

No "start a session". No forgetting where you were.

05

T: +44 (0)208 058 5951

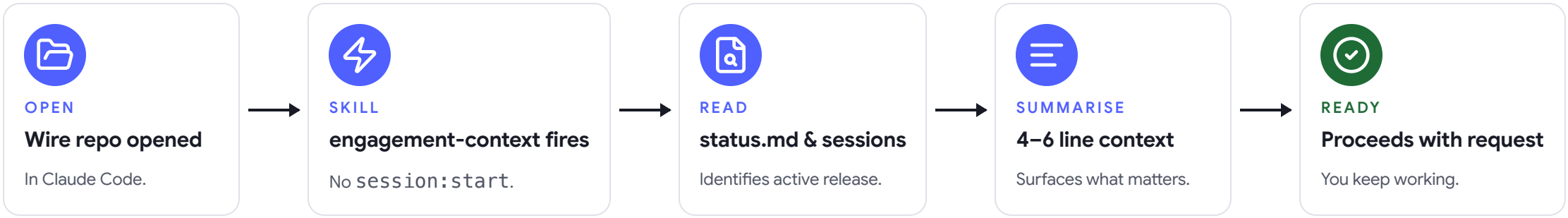
W: rittmananalytics.com

E: info@rittmananalytics.com

rittman analytics

STATE FLOW · V3.4.20+

From cold-start to ready, automatically.



T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

EXECUTION_LOG.MD

Your audit trail, session history, and handoff doc — in one file.

Every command and every skill activation appends a row. Nothing happens off the books.

TIMESTAMP	COMMAND / SKILL	RESULT	DETAIL
2026-05-12 09:15	`skill · engagement-context`	activated	Context loaded for 01-platform
2026-05-12 09:20	/wire:requirements-generate	complete	13 sections, 150 lines
2026-05-12 09:45	/wire:requirements-validate	pass	13 checks passed
2026-05-12 10:00	/wire:requirements-review	approved	Reviewed by Sarah Chen

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

OPTIONAL · STRUCTURED PLANNING

`/wire:plan` — agree the work before you start.

Use when you want to agree a focused work plan before starting a complex session.

```
/wire:plan 01-your-release
```

What it does

- Enters **Plan Mode** ✓
- Reads current release state ✓
- Proposes a 3–5 step plan ✓
- Waits for your approval before starting work ✓

Not required. Most sessions don't need it.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

PART SIX

Skills.

Ad-hoc development outside Wire commands — auto-activated when you need them.

T: +44 (0)208 058 5951

W: rittmananalytics.com

E: info@rittmananalytics.com

06

rittman analytics

SKILLS · HOW THEY WORK

Skills auto-activate — you don't invoke them.

A skill is a frontmatter-triggered context bundle. The trigger fires when its conditions match — you keep working, the AI just gets smarter.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

KEY SKILLS FOR WIRE ENGAGEMENTS

Six skills you'll meet in the first week.

SKILL	ACTIVATES WHEN	MAIN VALUE
`engagement-context`	`.wire/` present	Loads release state automatically.
`dbt-development`	dbt files	Conventions, naming, tests.
`lookml-content-authoring`	LookML files	Valid views, explores, dashboards.
`looker-dashboard-mockup`	"mockup" or "wireframe"	Pixel-accurate HTML Looker mockups.
`research`	Technical research	Saves and resurfaces prior findings.
`dignified-python`	Python files	Modern type syntax, LBYL, `pathlib`.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

PART SEVEN

Autopilot and advanced features.

When to let it run, when to keep your hands on the wheel.

07

T: +44 (0)208 058 5951

W: rittmananalytics.com

E: info@rittmananalytics.com

rittman analytics

AUTOPILOT

Run the full lifecycle, hands-off.

```
/wire:autopilot [sow-path]
```

Runs the full delivery lifecycle autonomously — discovery release (if no prior discovery), then every delivery phase for the chosen release type, with checkpoints at phase boundaries for human review.

WHEN TO USE

- Well-scoped SOW
- Familiar delivery type
- Accelerating the first pass

WHEN NOT TO

- First time with a client type
- Novel technical requirements
- Many clarifying questions needed

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

MCP INTEGRATIONS

Wire talks to the tools you already use.

INTEGRATION	WHAT IT ADDS
Fathom	Review commands pull relevant meeting context automatically.
Jira	Artifact lifecycle synced to Jira Sub-tasks.
Linear	Same sync, against Linear Projects and Issues.
Confluence / Notion	Generated artifacts published for client review.
Fivetran	Pipeline commands configure connectors via MCP.
dbt MCP	Development commands validate against live dbt state.

Configure with `/wire:mcp` — no JSON editing needed.

T: +44 (0)208 058 5951 W: rittmananalytics.com E: info@rittmananalytics.com

rittman analytics

EXTENDING WIRE

Add to it. Modify it. Make it yours.

ADD A COMMAND

Write a new spec in `wire/specs/`, add it to `build-packages.sh`, run the build.

MODIFY A COMMAND

Edit `wire/specs/<path>.md`. Takes effect on next invocation — no build needed for development.

ADD A SKILL

Create `wire/skills/<name>/SKILL.md` with frontmatter triggers. Add the `## On Activation` logging block.

CLIENT CONVENTIONS

Drop a client-level `SKILL.md` at `.wire/engagement/SKILL.md` with naming rules, chart types, measure patterns.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

PART EIGHT

Q&A and next steps.

Take it back to your laptop and try it on a real engagement.

08

T: +44 (0)208 058 5951

W: rittmananalytics.com

E: info@rittmananalytics.com

rittman analytics

NEXT STEPS

What to do between now and office hours.

- [ ] **Install Wire on your local machine today.**
  Both commands. Restart Claude Code. Confirm `/wire:help` works.
- [ ] **Create a test engagement using a recent SOW.**
  Any client you already know. Pick the release type that matches.
- [ ] **Run requirements + conceptual model end-to-end.**
  Before your next client call. Compare to what you'd have written by hand.
- [ ] **Bring your first real engagement to office hours.**
  Next week, same time. We'll work through it together.

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com

rittman analytics

RESOURCES

Where to find everything.

USER GUIDE	GitHub wiki · USER_GUIDE.md	QUICK REFERENCE	wire/docs/enablement/wire_developer_quick_reference.md
CHANGELOG	wire/docs/CHANGELOG.md	EXERCISES	wire/docs/enablement/wire_developer_exercises.md
SPEC SOURCE	wire/specs/	SKILLS SOURCE	wire/skills/

T: +44 (0)208 058 5951 W: rittmananalytics.com E: info@rittmananalytics.com

rittman analytics

THANK YOU · QUESTIONS?

Time to run your first Wire engagement.

Slack
#wire-framework

GitHub
github.com/rittmananalytics/wire/issues

Office hours
Tuesdays · 14:00 BST

T: +44 (0)208 058 5951
W: rittmananalytics.com
E: info@rittmananalytics.com